1. Simple Artificial Neuron

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
w = torch.tensor([0.2, 0.3, 0.4])
b = 0.5
y = torch.sigmoid(torch.dot(x, w) + b)
print("Neuron Output:", y.item())
```

2. Sigmoid and ReLU Activation Functions

```
import tensorflow as tf
import matplotlib.pyplot as plt
x = tf.range(-10, 11, dtype=tf.float32)
sigmoid = tf.sigmoid(x)
relu = tf.nn.relu(x)
plt.plot(x, sigmoid, label="Sigmoid")
plt.plot(x, relu, label="ReLU")
plt.legend()
plt.grid()
plt.show()
```

3. Feedforward Neural Network

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)
x = torch.tensor([[1., 2., 3.]])
y = model(x)
print("Output:", y.item())
```

4. Simple Neural Network Classifier

```
import tensorflow as tf
import numpy as np

x = np.array([[0,0], [0,1], [1,0], [1,1]],
dtype=float)
y = np.array([[0], [1], [1], [1]],
dtype=float)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(4,
activation='relu'),
    tf.keras.layers.Dense(1,
activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(x, y, epochs=200, verbose=0)
print("Accuracy:", model.evaluate(x, y,
verbose=0)[1])
```

5. Image Convolution Operations

```
import cv2
import numpy as np

img = np.array([[1,2,3], [4,5,6], [7,8,9]],
dtype=np.float32)
kernel = np.ones((3,3), dtype=np.float32)

result = cv2.filter2D(img, -1, kernel)
print(result)
```

6. Image Filters (Edge Detection, Blur)

```
import cv2
img = cv2.imread("image.jpg")

if img is None:
    print("Image not found!")
else:
    blur = cv2.GaussianBlur(img, (5,5), 0)
    edge = cv2.Canny(img, 100, 200)

    cv2.imshow("Original Image", img)
    cv2.imshow("Gaussian Blur", blur)
    cv2.imshow("Edge Detection", edge)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

7. CNN Feature Maps

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

img = torch.rand(1, 1, 5, 5)
conv = nn.Conv2d(1, 2, 3)
features = conv(img)

for i in range(2):
    plt.subplot(1, 2, i+1)
    plt.imshow(features[0,
i].detach().numpy(), cmap='gray')

plt.show()
```

8. Max Pooling and Average Pooling

```
import tensorflow as tf

x = tf.constant([[[[1.], [2.], [3.], [4.]],
                  [[5.], [6.], [7.], [8.]],
                  [[9.], [10.], [11.], [12.]],
                  [[13.], [14.], [15.],
[16.]]]])

max_pool = tf.nn.max_pool2d(x, 2, 2, 'VALID')
avg_pool = tf.nn.avg_pool2d(x, 2, 2, 'VALID')

print("Max Pooling:\n", max_pool.numpy())
print("Average Pooling:\n", avg_pool.numpy())
```

9. CNN for Image Classification

```
import tensorflow as tf

(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()

x_train = x_train[..., None] / 255.0
x_test = x_test[..., None] / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28,28,1)),
    tf.keras.layers.Conv2D(16, 3,
activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10,
activation='softmax')
])
model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=2)
print("Test Accuracy:", model.evaluate(x_test,
y_test)[1])
```

10. Pre-trained Model for Transfer Learning

```
import tensorflow as tf

base = tf.keras.applications.MobileNetV2(
    weights='imagenet', include_top=False,
input_shape=(32,32,3))
base.trainable = False

model = tf.keras.Sequential([
    base,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(10,
activation='softmax')
])

model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy')
model.fit(tf.random.normal((10,32,32,3)),
tf.range(10), epochs=1)

print("Transfer Learning Completed")
```

11. Basic RNN for Sequence Prediction

```
import torch
import torch.nn as nn

x = torch.tensor([[[1.], [2.], [3.]]])

rnn = nn.RNN(1, 5, batch_first=True)
fc = nn.Linear(5, 1)

out, hidden = rnn(x)
prediction = fc(out[:, -1, :])

print("Next Value:", prediction.item())
```

12. Reinforcement Learning

```
import numpy as np

q = np.zeros((3, 2))
for i in range(1000):
    s = 0
    while s != 2:
        a = np.random.randint(2)
        n = min(2, s+1) if a else max(0, s-1)
        r = 10 if n == 2 else 0
        q[s,a] += 0.1 * (r + 0.9 * max(q[n]) -
q[s,a])
        s = n
print("Q Table:\n", q)
```

13. K-Means Clustering

```
from sklearn.cluster import KMeans
import numpy as np

x = np.array([[1,2], [2,3], [3,4],
              [8,9], [9,10], [10,11]])

model = KMeans(n_clusters=2, random_state=0,
n_init=10)
model.fit(x)

print("Clusters:", model.labels_)
print("Centers:\n", model.cluster_centers_)
```

14. Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

x = load_iris().data

pca = PCA(n_components=2)
result = pca.fit_transform(x)

print("Original Shape:", x.shape)
print("Reduced Shape:", result.shape)
```

Program 1

Question:

Load an employee dataset and do basic exploration using pandas.

Code:

```
import pandas as pd

# Create employee dataset
data = {
    'EmpID': [101, 102, 103, 104, 105, 106, 107, 108],
    'Name': ['Arun', 'Bala', 'Charu', 'Divya',
             'Esha', 'Farhan', 'Gita', 'Hari'],
    'Department': ['HR', 'IT', 'IT', 'Finance',
                   'HR', 'IT', 'Finance', 'Sales'],
    'Age': [28, 34, 45, 29, 38, 41, 31, 26],
    'Salary': [35000, 52000, 75000, 48000,
               41000, 68000, 50000, 32000],
    'Experience': [3, 8, 20, 4, 12, 15, 6, 2]
}
df = pd.DataFrame(data)

print("First 5 rows:")
print(df.head())
print("\nShape (rows, columns):", df.shape)
print("\nColumn names:")
print(list(df.columns))
print("\nData types:")
print(df.dtypes)
print("\nStatistical summary:")
print(df.describe())
print("\nMissing values:")
print(df.isnull().sum())
print("\nEmployees per department:")
print(df['Department'].value_counts())
print("\nAverage salary by department:")
print(df.groupby('Department')['Salary'].mean())
```

Program 2

Question:
Load a dataset of students with missing age, duplicate records and outlier marks. Clean the dataset and display it.

Code:

```
import pandas as pd
import numpy as np

# Student dataset with missing age, duplicates and outlier marks
data = {
    'RollNo': [1, 2, 3, 4, 4, 5, 6, 7, 8, 8],
    'Name': ['Anu', 'Ben', 'Cara', 'Dev', 'Dev', 'Eva',
             'Fay', 'Gus', 'Hina', 'Hina'],
    'Age': [18, 19, np.nan, 20, 20, np.nan, 19, 18, 21, 21],
    'Marks': [78, 85, 69, 74, 74, 250, 81, 72, -40, -40]
}
df = pd.DataFrame(data)
print("Original dataset:")
print(df)
print("\nMissing values:\n", df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 1. Fill missing age with median
df['Age'] = df['Age'].fillna(df['Age'].median())

# 2. Remove duplicate records
df = df.drop_duplicates()

# 3. Remove outliers in Marks (valid range 0 - 100)
df = df[(df['Marks'] >= 0) & (df['Marks'] <= 100)]

# 4. Reset index and display
df = df.reset_index(drop=True)
df['Age'] = df['Age'].astype(int)
print("\nCleaned dataset:")
print(df)
print("\nShape after cleaning:", df.shape)
```

Program 3

Question:

Perform normalization and standardization on the iris dataset.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler, StandardScaler

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
print("Original data (first 5 rows):")
print(df.head())

# Normalization (Min-Max scaling to 0-1)
norm = MinMaxScaler()
df_norm = pd.DataFrame(norm.fit_transform(df), columns=df.columns)
print("\nNormalized data (first 5 rows):")
print(df_norm.head())
print("Min:\n", df_norm.min())
print("Max:\n", df_norm.max())

# Standardization (mean = 0, std = 1)
std = StandardScaler()
df_std = pd.DataFrame(std.fit_transform(df), columns=df.columns)
print("\nStandardized data (first 5 rows):")
print(df_std.head())
print("Mean:\n", df_std.mean().round(2))
print("Std:\n", df_std.std(ddof=0).round(2))
```

Program 4

Question:

Load the wine dataset, extract any attribute and normalize it.

Code:

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import MinMaxScaler

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
print("Attributes in wine dataset:")
for i, col in enumerate(df.columns, 1):
    print(i, col)

# Extract one attribute - alcohol
alcohol = df[['alcohol']]
print("\nOriginal 'alcohol' (first 10 values):")
print(alcohol['alcohol'].head(10).tolist())
print("Min =", alcohol['alcohol'].min())
print("Max =", alcohol['alcohol'].max())

# Normalize using Min-Max scaling
scaler = MinMaxScaler()
alcohol_norm = scaler.fit_transform(alcohol)
df['alcohol_normalized'] = alcohol_norm
print("\nNormalized 'alcohol' (first 10 values):")
print(df[['alcohol', 'alcohol_normalized']].head(10))
print("\nNew Min =", alcohol_norm.min())
print("New Max =", alcohol_norm.max())
```

Program 5

Question:

Predict profit based on advertising price using Linear Regression.

Code:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Advertising price (in thousands) and profit (in thousands)
data = {
    'Advertising': [10, 15, 20, 25, 30, 35, 40, 45, 50, 55],
    'Profit': [25, 34, 41, 52, 58, 69, 75, 86, 91, 103]
}
df = pd.DataFrame(data)
print(df)

X = df[['Advertising']]
y = df['Profit']

model = LinearRegression()
model.fit(X, y)

print("\nSlope (coefficient):", round(model.coef_[0], 4))
print("Intercept:", round(model.intercept_, 4))

y_pred = model.predict(X)
print("\nR2 score:", round(r2_score(y, y_pred), 4))
mse = mean_squared_error(y, y_pred)
print("Mean squared error:", round(mse, 4))

# Predict profit for a new advertising price
new_ad = pd.DataFrame({'Advertising': [60, 70]})
print("\nPredicted profit:")
for a, p in zip(new_ad['Advertising'], model.predict(new_ad)):
    print(f"Advertising = {a}  ->  Profit = {p:.2f}")
```

Program 6

Question:

Predict marks (Pass / Fail) based on study hours using Logistic Regression.

Code:

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# Study hours and marks; Result: 1 = Pass (marks >= 40), 0 = Fail
data = {
    'Hours': [0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4,
              4.5, 5, 5.5, 6, 6.5, 7, 8],
    'Marks': [10, 15, 22, 28, 35, 38, 45, 50,
              56, 62, 68, 74, 80, 86, 95]
}
df = pd.DataFrame(data)
df['Result'] = (df['Marks'] >= 40).astype(int)
print(df)

X = df[['Hours']]
y = df['Result']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Predict result for new study hours
for h in [1.0, 3.0, 4.0, 6.0]:
    res = model.predict(pd.DataFrame({'Hours': [h]}))[0]
    prob = model.predict_proba(pd.DataFrame({'Hours': [h]}))[0][1]
    print(f"Hours = {h}: {'Pass' if res == 1 else 'Fail'} "
          f"(Pass probability = {prob:.2f})")
```

Program 7

Question:

Load the iris dataset and predict the species name based on any attribute, using KNN.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
# Use one attribute: petal length (cm)
X = iris.data[:, [2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Predict species for new petal lengths
for length in [1.4, 4.0, 5.8]:
    p = knn.predict([[length]])[0]
    print(f"Petal length = {length} cm -> {iris.target_names[p]}")
```

Program 8

Question:

Load the iris dataset and predict the species name based on any attribute, using Decision Tree.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
# Use one attribute: petal length (cm)
X = iris.data[:, [2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

tree = DecisionTreeClassifier(max_depth=3, random_state=0)
tree.fit(X_train, y_train)

y_pred = tree.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("\nDecision Tree rules:")
print(export_text(tree, feature_names=['petal length (cm)']))

# Predict species for new petal lengths
for length in [1.4, 4.0, 5.8]:
    p = tree.predict([[length]])[0]
    print(f"Petal length = {length} cm -> {iris.target_names[p]}")
```

Program 9

Question:
Load the iris dataset and predict the species name based on any attribute, using Naive Bayes.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
# Use one attribute: petal length (cm)
X = iris.data[:, [2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

nb = GaussianNB()
nb.fit(X_train, y_train)

y_pred = nb.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Predict species for new petal lengths
for length in [1.4, 4.0, 5.8]:
    p = nb.predict([[length]])[0]
    print(f"Petal length = {length} cm -> {iris.target_names[p]}")
```

Program 10

Question:

Load the iris dataset and predict the species name based on any attribute, using K-Means Clustering.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

iris = load_iris()
# Use one attribute: petal length (cm)
X = iris.data[:, [2]]

kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = kmeans.fit_predict(X)

print("Cluster centres (petal length):")
print(kmeans.cluster_centers_.round(3).ravel())
print("\nInertia:", round(kmeans.inertia_, 4))

# Compare clusters with actual species
species = [iris.target_names[i] for i in iris.target]
df = pd.DataFrame({'Species': species, 'Cluster': labels})
print("\nCluster vs Species:")
print(pd.crosstab(df['Species'], df['Cluster']))

# Predict cluster for new petal lengths
for length in [1.4, 4.0, 5.8]:
    c = kmeans.predict([[length]])[0]
    print(f"Petal length = {length} cm -> Cluster {c}")
```